

Bigrams: Bigrams in the given input string can be obtained using `nltk.bigrams()`. This function actually returns a generator object. The code fragment given below illustrates the same:

```
>>> s="open source for you"
>>> nltk.bigrams(s)
<generator object bigrams at 0x7fc49a8025f0>
```

Output can be obtained as a list by casting this generator to *list* as shown below.

```
>>> list(nltk.bigrams(s))
[('o', 'p'), ('p', 'e'), ('e', 'n'), ('n', ' '), (' ', 's'), ('s', 'o'), ('o', 'u'), ('u', 'r'), ('r', 'c'), ('c', 'e'), ('e', ' '), (' ', 'f'), ('f', 'o'), ('o', 'r'), ('r', ' '), (' ', 'y'), ('y', 'o'), ('o', 'u')]
```

You may feel that the output is not in the form desired because you were expecting word bigrams. In that case, the desired result can be obtained by tokenising or splitting the input into words before passing it to `bigrams()`.

```
>>> list(nltk.bigrams(nltk.word_tokenize(s)))
[('open', 'source'), ('source', 'for'), ('for', 'you')]
```

Now the result seems perfect.

Collocations: `nltk.collocations()` lists bigrams occurring frequently in the input. The statement given below illustrates its result when applied to *text1* in the NLTK corpus:

```
>>> text1.collocations(num=5)
Sperm Whale; Moby Dick; White Whale; old man; Captain Ahab
```

The parameter *num* specifies the number of results that should be displayed. In the above example, as specified, the output contains only five bigrams. If we call the function without any parameter, all the frequent bigrams will be displayed.

Vocabulary of a given input: `set()` in NLTK will do the job. The output of this function preserves the property of a set, i.e., duplicates are not allowed in a set. An example is given below:

```
>>> s="open source for you serves you open source related news"
>>> set(nltk.word_tokenize(s))
set(['for', 'related', 'source', 'serves', 'news', 'you', 'open'])
```

If you do not tokenise 's' before applying `set()`, the character set in 's' will be the result.

Feature structures

Constraints on natural language input can be better represented

by exploiting features. NLTK provides the feature structure to map between feature identifiers and feature values, i.e., it represents a set of feature-value pairs. Feature identifiers are also called feature names. Feature values can be a simple string, *int* or nested feature values. It is useful when we wish to apply a similar treatment to feature-value pairs.

The general form of a feature structure is `[feature1=value1, feature2=value2, .....]`. Before use, it should be imported.

```
>>> from nltk.feature import FeatStruct
```

The statements given below will generate three feature structures -- *feat1*, *feat2* and *feat3*.

```
>>> feat1 = FeatStruct(number='singular', person=3)
>>> feat2=FeatStruct(agr=feat1)
>>> feat3=FeatStruct(cat='VP',agr=feat1, head=feat2)
>>> feat3
[agr=(1)[number='singular', person=3], cat='VP', head=[agr->(1)]]
```

Here, *feat1* contains two attributes — *number* and *person*. We can see that *feat3*'s attribute, *head*, has nested value.

A feature structure can be a feature list or a feature dictionary, which is automatically decided by *FeatStruct* depending on the type of argument passed to it. In a feature list, the feature identifier will be an integer. An example is shown below:

```
>>> type(FeatStruct([1,2,3]))
<class 'nltk.feature.FeatList'>
>>> type(FeatStruct(sent=feat3))
<class 'nltk.feature.FeatDict'>
```

A lot more can be done with NLTK. This article is just an introduction to it. A complete reference to natural language processing using Python can be found in the book 'Natural Language Processing with Python' by Steven Bird, Ewan Klein and Edward Loper.  **END**

References

- [1] <https://stackoverflow.com/questions/4634787/freqdist-with-nltk>
- [2] <https://stackoverflow.com/questions/10467024/how-do-i-create-my-own-nltk-text-from-a-text-file>
- [3] <https://docs.python.org/2.3/whatsnew/node7.html>
- [4] www.textminingonline.com
- [5] <https://www.nltk.org>
- [6] <https://stackoverflow.com/questions/37651057/generate-bigrams-with-nltk>

By: Sharon Sunny

The author is an assistant professor at Amaljiyothi College of Engineering, Kerala. She can be reached at ssharon099@gmail.com.